```
"""
StegSim report.py — Markdown run report generator.
Produces REPORT.md explaining what happened, why, and what it means.
"""

import json
from datetime import datetime, timezone
from pathlib import Path


def load_jsonl(path: str) -> list:
    records = []
    p = Path(path)
    if not p.exists():
        return records
    with open(p) as f:
        for line in f:
            line = line.strip()
            if line:
                try:
                    records.append(json.loads(line))
                except Exception:
                    pass
    return records


def generate_report(run_dir: str, cfg_dict: dict, summary: dict) -> str:
    """
    Generate REPORT.md from run artifacts.
    Returns the report text and writes it to run_dir/REPORT.md.
    """
    run_path = Path(run_dir)
    metrics  = load_jsonl(str(run_path / "metrics.jsonl"))
    receipts = load_jsonl(str(run_path / "receipts.jsonl"))

    # Key events
    first_warning_step      = summary.get("first_warning_step", "N/A")
    first_inadmissible_step = summary.get("first_inadmissible_step", "N/A")
    min_viability           = summary.get("min_viability", "N/A")
    initial_viability       = summary.get("initial_viability", "N/A")
    final_viability         = summary.get("final_viability", "N/A")

    # First major denial
    first_deny = next(
```

```
    (r for r in receipts
     if r.get("decision") in ("DENY", "FAIL_CLOSED")
     and not r.get("local_policy_valid", True) is False),
    None
)
first_global_deny = next(
    (r for r in receipts
     if r.get("decision") in ("DENY", "FAIL_CLOSED")
     and r.get("local_policy_valid") is True),
    None
)

# Aggregate stats
total         = summary.get("total_proposals", 0)
allowed       = summary.get("accepted_count", 0)
denied        = summary.get("denied_count", 0)
flagged       = summary.get("flagged_count", 0)
fail_closed   = summary.get("fail_closed_count", 0)
lg_disagree   = summary.get("peak_local_global_disagreement_rate", 0)

# Viability trajectory from metrics
viab_series = [(m["step"], m["viability_composite"]) for m in metrics]
inflection   = None
for i in range(1, len(viab_series)):
    if viab_series[i][1] < 0.45 and viab_series[i-1][1] >= 0.45:
        inflection = viab_series[i][0]
        break

scenario = cfg_dict.get("scenario", "unknown")
run_id   = cfg_dict.get("run_id", "unknown")
seed     = cfg_dict.get("seed", 0)
n_agents = cfg_dict.get("agents", 0)
n_steps  = cfg_dict.get("steps", 0)

lines = [
    f"# StegSim Run Report",
    f"",
    f"**Scenario:** `{scenario}`  ",
    f"**Run ID:** `{run_id}`  ",
    f"**Seed:** `{seed}`  ",
    f"**Agents:** {n_agents:,}  ",
    f"**Steps:** {n_steps}  ",
    f"**Generated:** {datetime.now(timezone.utc).isoformat()}",
    f"",
    f"---",
    f"",
    f"## Core Demonstration",
```

```
    f"",
    f"> **A system can remain locally compliant while becoming globally ungov
    f"",
    f"This run demonstrates that local policy checks remained passing while",
    f"global viability collapsed. The peak local/global disagreement rate",
    f"was **{lg_disagree:.1%}** — meaning at peak, that fraction of proposals
    f"passed local policy while failing global admissibility.",
    f"",
    f"---",
    f"",
    f"## Initial Conditions",
    f"",
    f"| Parameter | Value |",
    f"|---|---|",
    f"| Trust mean | {cfg_dict.get('initial_conditions', {}).get('trust_mean'
    f"| Authority mean | {cfg_dict.get('initial_conditions', {}).get('authori
    f"| Resource balance mean | {cfg_dict.get('initial_conditions', {}).get('
    f"| Alignment mean | {cfg_dict.get('initial_conditions', {}).get('alignme
    f"| Policy freshness | {cfg_dict.get('initial_conditions', {}).get('polic
    f"| Initial viability | {initial_viability:.4f} if isinstance(initial_via
    f"",
    f"---",
    f"",
    f"## Drift Conditions",
    f"",
    f"| Drift Parameter | Per-Step Rate |",
    f"|---|---|",
]

drift = cfg_dict.get("drift", {})
for k, v in drift.items():
    lines.append(f"| {k.replace('_', ' ').title()} | {v} |")

lines += [
    f"",
    f"---",
    f"",
    f"## Transition Summary",
    f"",
    f"| Metric | Value |",
    f"|---|---|",
    f"| Total proposals | {total:,} |",
    f"| Accepted (ALLOW) | {allowed:,} ({allowed/total:.1%} if total else 'N/
    f"| Denied (DENY) | {denied:,} ({denied/total:.1%} if total else 'N/A') |
    f"| Flagged (FLAG) | {flagged:,} |",
    f"| Fail-closed | {fail_closed:,} |",
    f"",
```

```
    f"---",
    f"",
    f"## Viability Trajectory",
    f"",
    f"| Event | Step | Viability |",
    f"|---|---|---|",
    f"| Initial | 0 | {initial_viability:.4f} if isinstance(initial_viability
    f"| First warning threshold crossed | {first_warning_step} | — |",
    f"| First inadmissible threshold crossed | {first_inadmissible_step} | —
    f"| Inflection to degraded (<0.45) | {inflection or 'N/A'} | — |",
    f"| Minimum viability | — | {min_viability:.4f} if isinstance(min_viabili
    f"| Final viability | {n_steps} | {final_viability:.4f} if isinstance(fin
    f"",
    f"---",
    f"",
    f"## First Global Governance Denial",
    f"",
]

if first_global_deny:
    lines += [
        f"At step **{first_global_deny.get('step')}**, agent `{first_global_d
        f"proposed an action that **passed local policy but failed global adm
        f"",
        f"- **Decision:** `{first_global_deny.get('decision')}`",
        f"- **Reason:** {first_global_deny.get('reason')}",
        f"- **Local policy valid:** {first_global_deny.get('local_policy_vali
        f"- **Authority valid (current state):** {first_global_deny.get('auth
        f"- **Viability before:** {first_global_deny.get('viability_before',
        f"- **Viability projected:** {first_global_deny.get('viability_after_
        f"- **Authority staleness:** {first_global_deny.get('authority_stalen
        f"",
        f"This is the key demonstration: **the agent had valid local credenti
        f"but its authority was stale, derived from an older system state tha
        f"longer supported the proposed transition under current conditions."
    ]
else:
    lines += [
        f"No local-pass / global-fail transitions were recorded in this run."
        f"Consider increasing drift rates or running more steps.",
    ]

lines += [
    f"",
    f"---",
    f"",
    f"## Why Local Compliance Was Insufficient",
```

```
    f"",
    f"The simulation demonstrates a governance failure mode common to:",
    f"distributed AI systems, financial markets, regulatory networks,",
    f"and any complex system where authority is inherited rather than",
    f"re-derived from current state.",
    f"",
    f"**Local governance checked:**",
    f"- Does the agent have minimum trust? ✓",
    f"- Does the agent have a credential? ✓",
    f"- Is the action within the agent's declared scope? ✓",
    f"",
    f"**Global governance additionally checked:**",
    f"- Is the agent's authority current (not stale)? ← **This failed**",
    f"- Does the transition preserve ecosystem viability? ← **This failed**",
    f"- Does the agent's autonomy stay within legitimacy capacity? ← **This f
    f"",
    f"The local checks passed. The system appeared compliant.",
    f"The global checks revealed the system was already in collapse.",
    f"",
    f"---",
    f"",
    f"## StegVerse Governance Claim",
    f"",
    f"This run provides executable evidence for the StegVerse thesis:",
    f"",
    f"> **Governance failure is often not a failure of policy representation,
    f"> **but a failure to preserve viability under state mutation.**",
    f"",
    f"The GOD geometry's FAIL-CLOSED region (Y) is not an edge case.",
    f"It is the normal endpoint of any system where authority is inherited",
    f"rather than re-derived at commit against current state.",
    f"",
    f"---",
    f"",
    f"## Receipts",
    f"",
    f"Full receipt trail: `receipts.jsonl` ({len(receipts):,} records)",
    f"Full metrics trail: `metrics.jsonl` ({len(metrics):,} records)",
    f"",
    f"*Source of truth: receipts.jsonl. This report publishes the surface;*",
    f"*it does not generate receipts.*",
]

# Fix f-string formatting issues in table cells
report_text = "\n".join(lines)
# Replace the broken conditional expressions
if isinstance(initial_viability, float):
```

```
    report_text = report_text.replace(
        f"{initial_viability:.4f} if isinstance(initial_viability, float) els
        f"{initial_viability:.4f}"
    )
if isinstance(min_viability, float):
    report_text = report_text.replace(
        f"{min_viability:.4f} if isinstance(min_viability, float) else min_vi
        f"{min_viability:.4f}"
    )
if isinstance(final_viability, float):
    report_text = report_text.replace(
        f"{final_viability:.4f} if isinstance(final_viability, float) else fi
        f"{final_viability:.4f}"
    )

report_path = run_path / "REPORT.md"
with open(report_path, "w") as f:
    f.write(report_text)

return report_text
```